

1^ο Εργαστήριο Αντικειμενοστραφούς Προγραμματισμού

Στόχος της ενότητας αυτής είναι η γνωριμία και εξοικείωση με την Standard Library της C++ και κυρίως με το μέλλος “string”.

String:

Τα strings είναι πολύπλοκοι τύποι δεδομένων (αντικείμενα) που αναπαριστούν ακολουθίες χαρακτήρων, με απλά λόγια είναι ένας πολύ αποδοτικός τρόπος για να αναπαραστήσουμε κείμενο. Επιπλέον οι υλοποιημένες μέθοδοι-συναρτήσεις μας δίνουν ένα μεγάλο εύρος από εργαλεία για να μπορούμε να επεξεργαστούμε αυτές τις ακολουθίες χαρακτήρων. Πρακτικά μας δίνονται ένας μεγάλος αριθμός από μεθόδους/συναρτήσεις για να αντλήσουμε πληροφορία για το μέγεθος και τον καταληφθέντα χώρο μνήμης ενός string, αλλά και για να το επεξεργαστούμε. Δηλαδή να προσπελαύνουμε κάποιους μόνο χαρακτήρες, σε διάφορες θέσεις μέσα στο string, να τους αντικαταστήσουμε, απλά να τους διαγράψουμε ή ακόμη και να προσθέσουμε νέους χαρακτήρες στις θέσεις αυτές. Μεταξύ των άλλων μας παρέχονται και εργαλεία αναζήτησης μεμονωμένων ή και συνδυασμών χαρακτήρων μέσα στο string, αλλά και η δυνατότητα να συγκρίνουμε δύο string ή μέρη αυτών.

Βήμα 1^ο - Ανάθεση τιμής - Χωρητικότητα (Capacity) – Πρόσβαση σε στοιχεία

Στο βήμα αυτό θα διερευνήσουμε τις παρακάτω μεθόδους/συναρτήσεις

<code>string& string::operator= (const string& str);</code>	Εκχωρεί μία νέα τιμή στο string, αντικαθιστώντας την τρέχουσα.
<code>size_t string::size () const;</code>	Επιστρέφει το μήκος του string σε bytes. Αυτός είναι ο αριθμός των πραγματικών bytes που απαιτούν τα περιεχόμενα του string, ο οποίος δεν είναι απαραίτητα ίσος με την τιμή του capacity.
<code>size_t string::length () const;</code>	Τόσο το <code>string::size</code> όσο και η <code>string::length</code> είναι συνώνυμες και επιστρέφουν την ίδια τιμή.
<code>size_t string::max_size () const;</code>	Επιστρέφει το μέγιστο μήκος που μπορεί να φτάσει το string. Αυτό είναι το μέγιστο δυνητικό μήκος που μπορεί να φτάσει το string λόγω γνωστών περιορισμών από το σύστημα ή τη βιβλιοθήκη, χωρίς να είναι εγγυημένο ότι το string μπορεί να φτάσει αυτό το μήκος.
<code>void string::resize (size_t n);</code>	Επαναπροσδιορίζει το μήκος του string σε <i>n</i> χαρακτήρες. Αν το <i>n</i> είναι μικρότερο από το τρέχον, η του τιμή του string μειώνεται στους πρώτους <i>n</i> χαρακτήρες, απομακρύνοντας τους χαρακτήρες μετά τον <i>n</i> -στο. Αν το <i>n</i> είναι μεγαλύτερο από τρέχον μήκος του string, το τρέχον περιεχόμενο επεκτείνεται, εισάγοντας στο τέλος όσους χαρακτήρες χρειάζονται για να γίνει το μήκος του string ίσο με <i>n</i> .
<code>size_t string::capacity () const;</code>	Επιστρέφει το μέγεθος του χώρου που καταλαμβάνει το string σε bytes. Το <code>string::capacity</code> δεν

	είναι απαραίτητα ίσο με το μέγεθος της <code>string::length</code> . Μπορεί να είναι ίση ή μεγαλύτερη, με τον extra χώρο να επιτρέπει στο αντικείμενο την βελτιστοποίηση των λειτουργιών του όταν νέοι χαρακτήρες προστίθενται στο string.
<code>void string::clear ();</code>	Καθαρίζει / αδειάζει τα περιεχόμενα του string, καθιστώντας το ένα άδειο string (με μέγεθος 0 χαρακτήρες)
<code>bool string::empty () const;</code>	Επιστρέφει 0 αν το string είναι άδειο, αν δηλαδή το μήκος είναι 0, διαφορετικά επιστρέφει 1. Η συνάρτηση αυτή δεν τροποποιεί την τιμή του string με κανένα τρόπο.
<code>char& string::operator[] (size_t pos);</code>	Επιστρέφει μία αναφορά στον χαρακτήρα που βρίσκεται σε μία συγκεκριμένη θέση (την <i>pos</i>) στο string. Αν η <i>pos</i> είναι ίση με το μήκος του string, η συνάρτηση επιστρέφει μία αναφορά σε κενό χαρακτήρα ('\0')
<code>char& string::at (size_t pos);</code>	Η λειτουργία της είναι σχεδόν πανομοιότυπη με αυτή της <code>string::operator[]</code> . Η κύρια διαφορά τους είναι στον τρόπο σύνταξης.

Πληκτρολογήστε τον παρακάτω κώδικα και ελέγξτε το αποτέλεσμα.

```

1  #include <iostream>
2  #include <string>
3
4  int main ()
5  {
6      std::string str0, str1;
7      str1 = "Hello world.";
8      std::cout << "Size of str0: " << str0.size() << " and size of str1: " << str1.size() << "\n";
9      std::cout << "Length of str0: " << str0.length() << " and length of str1: " << str1.length() << "\n";
10     std::cout << "Capacity of str0: " << str0.capacity() << " and capacity of str1: " << str1.capacity() << "\n";
11     std::cout << "Max_size of str0: " << str0.max_size() << " and max_size of str1: " << str1.max_size() << "\n";
12     return 0;
13 }
```

Προσθέστε τον παρακάτω κώδικα πριν την "return 0;" και ελέγξτε το αποτέλεσμα:

```
12     bool b10=str0.empty();
13     bool b11=str1.empty();
14     if (str0.empty()){
15         std::cout << "str0 is empty!\n";
16     }else{
17         std::cout << "str0 is not empty!\n";
18     }
19     if (str1.empty()){
20         std::cout << "str1 is empty!\n";
21     }else{
22         std::cout << "str1 is not empty!\n";
23     }
24     str0=str1;
25     str1.clear();
26
27     if (str0.empty()){
28         std::cout << "Now str0 is empty!\n";
29     }else{
30         std::cout << "Now str0 is not empty!\n";
31     }
32     if (str1.empty()){
33         std::cout << "Now str1 is empty!\n";
34     }else{
35         std::cout << "Now str1 is not empty!\n";
36     }
37     return 0;
38 }
```

Προσθέστε τον παρακάτω κώδικα πριν την "return 0;" και ελέγξτε το αποτέλεσμα:

```
37     std::cout << "Ektupwsn me tn xrnsn tns []:\n";
38     for (int i=0; i<str0.length(); ++i)
39     {
40         std::cout << str0[i];
41     }
42     std::cout << "\nEktupwsn me tn xrnsn tns at:\n";
43     for (int i=0; i<str0.length(); ++i)
44     {
45         std::cout << str0.at(i);
46     }
```

Βήμα 2^ο – Τροποποίηση

Στο βήμα αυτό θα διερευνήσουμε τις παρακάτω μεθόδους/συναρτήσεις

<code>string string::operator+ (const string& lhs, const string& rhs);</code>	Επιστρέφει ένα νεοκατασκευασμένο string, με την τιμή του να αποτελείται από την συνένωση των χαρακτήρων του <i>lhs</i> ακολουθούμενοι από αυτούς του <i>rhs</i> .
<code>string& string::operator+= (const string& str);</code>	Επεκτείνει το string προσθέτοντας του στο τέλος του επιπλέον χαρακτήρες. Η κύρια διαφορά με το <i>string::operator+</i> είναι ότι το τελευταίο μπορεί να επαναληφθεί πολλές φορές στην ίδια έκφραση, όχι όμως και το <i>string::operator+=</i> .
<code>string& string::append (const string& str);</code>	Η λειτουργία του είναι πανομοιότυπη με αυτή του <i>string::operator+=</i> , αλλάζει μόνο ο τρόπος σύνταξης.
<code>void string::pop_back ();</code>	Διαγράφει τον τελευταίο χαρακτήρα του string, μειώνοντας το μήκος του string κατά ένα.
<code>void string::push_back (char c);</code>	Προσθέτει έναν χαρακτήρα <i>c</i> στο τέλος του string, αυξάνοντας το μήκος του κατά ένα.
<code>string& string::assign (const string& str);</code>	Αναθέτει μία καινούργια τιμή στο string, αντικαθιστώντας την παλιά.
<code>string& string::insert (size_t pos, const string& str);</code> <code>string& string::insert (size_t pos, const string& str, size_t subpos, size_t sublen);</code> <code>string& string::insert (size_t pos, const char* s, size_t n);</code> <code>string& string::insert (size_t pos, size_t n, char c);</code>	Εισάγει νέους χαρακτήρες σε υπάρχον string, ακριβώς πριν τον χαρακτήρα που υποδεικνύεται από συγκεκριμένη δηλωθείσα θέση <i>pos</i> .
<code>string& string::erase (size_t pos = 0, size_t len = npos);</code>	Αφαιρεί μέρος του string, μειώνοντας το μήκος του ανάλογα.
<code>string& string::replace (size_t pos, size_t len, const string& str);</code> <code>string& string::replace (size_t pos, size_t len, const string& str, size_t subpos, size_t sublen);</code> <code>string& string::replace (size_t pos, size_t len, const char* s, size_t n);</code> <code>string& string::replace (size_t pos, size_t len, size_t n, char c);</code>	Αντικαθιστά μέρος του string που ξεκινά από δηλωθείσα θέση <i>pos</i> και για συγκεκριμένο αριθμό <i>len</i> χαρακτήρων με νέο περιεχόμενο.
<code>void string::swap (string& x, string& y);</code>	Ανταλλάσει το περιεχόμενο δύο strings <i>x</i> και <i>y</i> .

Τροποποιήστε κατάλληλα τον αρχικό κώδικα (copy/paste/amend) ώστε να προκύψει ο παρακάτω κώδικας:

```
1  #include <iostream>
2  #include <string>
3
4  int main ()
5  {
6      std::string str01, str02, str03, str04, str05;
7      str01="avti";
8      str02.assign("para");
9      str03.assign("8esn");
10     str04=str01+str02+str03;
11     std::cout << "str04(str01+tr02+str03): " << str04 << "\n";
12     str01.swap(str02);
13     str05.append(str01);
14     str05.append(str03);
15     str02+=str05;
16     std::cout << "str02(str02+=str05): " << str02 << "\n";
17     str02.erase(0,4);
18     std::cout << "str02 after erasing from pos 0, 4 chars: " << str02 << "\n";
19
20     str01.clear();
21     int k=0;
22     for (int i=0; i<32;i++){
23         for (int j=0; j<16;j++){
24             str01=(k);
25             std::cout << "k " << k << " = " << str01 << ", ";
26             k++;
27         }
28         std::cout << "\n";
29     }
30     str01.clear();
31     std::cout << "str01.clear(): " << str01 << "\n";
32     for (int i=1; i<257;i++){
33         str01+=(i);
34     }
35     std::cout << "str01: " << str01 << "\n";
36     return 0;
37 }
```

Εκτελέστε και ελέγξτε το αποτέλεσμα.

Τροποποιήστε κατάλληλα τον παραπάνω κώδικα (copy/paste/amend) ώστε να πάρει την παρακάτω μορφή:

```
1  #include <iostream>
2  #include <string>
3
4  int main ()
5  {
6      std::string str01="to be question";
7      std::string str02="the ";
8      std::string str03="or not to be";
9
10     str01.insert(6,str02);           // to be (the )question
11     std::cout << str01 << '\n';
12     str01.insert(6,str03,3,4);       // to be (not )the question
13     std::cout << str01 << '\n';
14     str01.insert(10,"that is cool",8); // to be not (that is )the question
15     std::cout << str01 << '\n';
16     str01.insert(10,"to be ");       // to be not (to be )that is the question
17     std::cout << str01 << '\n';
18     str01.insert(15,1,':');          // to be not to be(:) that is the question
19     std::cout << str01 << '\n';
20
21     std::string base="this is a test string.";
22     str02="n example";
23     str03="sample phrase";
24
25     // replace signatures used in the same order as described above:
26
27     str01=base;                      // "this is a test string."
28     std::cout << str01 << '\n';
29     str01.replace(9,5,str02);         // "this is an example string." (1)
30     std::cout << str01 << '\n';
31     str01.replace(19,6,str03,7,6);    // "this is an example phrase." (2)
32     std::cout << str01 << '\n';
33     str01.replace(8,10,"just a");     // "this is just a phrase." (3)
34     std::cout << str01 << '\n';
35     str01.replace(8,6,"a shorty",7);  // "this is a short phrase." (4)
36     std::cout << str01 << '\n';
37     str01.replace(22,1,3, '!');       // "this is a short phrase!!!" (5)
38     std::cout << str01 << '\n';
39     return 0;
40 }
```

Εκτελέστε και ελέγξτε το αποτέλεσμα.

Βήμα 3^ο - Λειτουργίες

Στο βήμα αυτό θα διερευνήσουμε τις παρακάτω μεθόδους/συναρτήσεις

<code>size_t string::copy (char* s, size_t len, size_t pos = 0) const;</code>	Αντιγράφει μία ακολουθία από χαρακτήρες από το υπάρχον string σε ένα array στο οποίο δείχνει ο s . Η ακολουθία αυτή περιέχει len χαρακτήρες, ξεκινώντας από αυτόν που βρίσκεται στην θέση pos . Η συνάρτηση αυτή δεν προσθέτει στο τέλος του αντιγραφόμενου περιεχομένου έναν άδειο χαρακτήρα.
<code>size_t string::find (const string& str, size_t pos = 0) const;</code> <code>size_t string::find (const char* s, size_t pos, size_t n) const;</code>	Ψάχνει το string για την πρώτη εμφάνιση της ακολουθίας που καθορίζεται από τα ορίσματα. Όταν ορίζεται το pos , η αναζήτηση περιλαμβάνει μόνο χαρακτήρες στην θέση pos και μετά από αυτή, αγνοώντας ότι υπάρχει πριν την θέση pos .
<code>size_t string::find_first_of (const string& str, size_t pos = 0) const;</code>	Ψάχνει στο string για τον πρώτο χαρακτήρα που ταιριάζει σε οποιοδήποτε από τους χαρακτήρες που καθορίζονται στα ορίσματα. Όταν ορίζεται το pos , η αναζήτηση περιλαμβάνει μόνο χαρακτήρες στην θέση pos και μετά από αυτή, αγνοώντας ότι υπάρχει πριν την θέση pos .
<code>size_t string::find_last_of (const string& str, size_t pos = npos) const;</code>	Ψάχνει στο string για τον τελευταίο χαρακτήρα που ταιριάζει σε οποιοδήποτε από τους χαρακτήρες που καθορίζονται στα ορίσματα. Όταν ορίζεται το pos , η αναζήτηση περιλαμβάνει μόνο χαρακτήρες στην θέση pos και πριν από αυτή, αγνοώντας ότι υπάρχει μετά την θέση pos .
<code>size_t string::find_first_not_of (const string& str, size_t pos = 0) const;</code>	Ψάχνει στο string για τον πρώτο χαρακτήρα που δεν ταιριάζει σε οποιοδήποτε από τους χαρακτήρες που καθορίζονται στα ορίσματα. Όταν ορίζεται το pos , η αναζήτηση περιλαμβάνει μόνο χαρακτήρες στην θέση pos και μετά από αυτή, αγνοώντας ότι υπάρχει πριν την θέση pos .

<pre>size_t string::find_last_not_of (const string& str, size_t pos = npos) const;</pre>	Ψάχνει στο string για τον τελευταίο χαρακτήρα που δεν ταιριάζει σε οποιοδήποτε από τους χαρακτήρες που καθορίζονται στα ορίσματα. Όταν ορίζεται το <i>pos</i>, η αναζήτηση περιλαμβάνει μόνο χαρακτήρες στην θέση <i>pos</i> και πριν από αυτή, αγνοώντας ότι υπάρχει μετά την θέση <i>pos</i>.
<pre>string string::substr (size_t pos = 0, size_t len = npos) const;</pre>	Επιστρέφει ένα νεοκατασκευασμένο string με το περιεχόμενό του να αρχικοποιείται από ένα αντίγραφο από ένα μέρος του αρχικού string. Το μέρος του αρχικού string ξεκινά από την θέση <i>pos</i> και εκτείνεται για <i>len</i> χαρακτήρες (ή μέχρι το τέλος του, όποιο προκύψει πρώτο) στο αρχικό string.
<pre>int string::compare (const string& str) const; int string::compare (size_t pos, size_t len, const string& str) const;</pre>	Συγκρίνει το περιεχόμενο δύο strings για την ακολουθία χαρακτήρων που καθορίζεται από τα ορίσματα. Όταν ορίζονται τα <i>pos</i> και <i>len</i> το αρχικό string συγκρίνεται με την ακολουθία που προκύπτει από το αρχικό string και ξεκινά από την θέση <i>pos</i> και για <i>len</i> χαρακτήρες.

Τροποποιήστε κατάλληλα τον παραπάνω κώδικα (copy/paste/amend) ώστε να πάρει την παρακάτω μορφή:

```
1  #include <iostream>
2  #include <string>
3
4  int main ()
5  {
6      char buffer[20];
7      std::string str ("There are two needles in this haystack with needles.");
8      std::size_t length = str.copy(buffer,6,14);
9      buffer[length]='\0';
10     std::cout << "buffer contains: " << buffer << '\n';
11     std::string str2=buffer;
12     std::string str3=str.substr (14,6);
13     if (str2.compare(str3) != 0)
14         std::cout << str2 << " is not " << str3 << '\n';
15     else
16         std::cout << str2 << " is equal to " << str3 << '\n';
17
18     std::cout << "str2 contains: " << str2 << '\n';
19     std::size_t found = str.find(str2);
20     if (found!=std::string::npos)
21         std::cout << "first 'needle' found at: " << found << '\n';
22
23     found=str.find("needles are small",found+1,6);
24     if (found!=std::string::npos)
25         std::cout << "second 'needle' found at: " << found << '\n';
26
27     found=str.find("haystack");
28     if (found!=std::string::npos)
29         std::cout << "'haystack' also found at: " << found << '\n';
30
31     found=str.find('.');
32     if (found!=std::string::npos)
33         std::cout << "Period found at: " << found << '\n';
34
35     std::cout << str << '\n';
36
37     found = str.find_first_of("aeiou");
38     while (found!=std::string::npos)
39     {
40         str[found]='*';
41         found=str.find_first_of("aeiou",found+1);
42     }
43
44     std::cout << str << '\n';
45     found = str.find_first_not_of("abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ ");
46
47     if (found!=std::string::npos)
48     {
49         std::cout << "The first non-alphabetic character is " << str[found];
50         std::cout << " at position " << found << '\n';
51     }
52     return 0;
53 }
```

Εκτελέστε και ελέγξτε το αποτέλεσμα.

Τροποποιήστε κατάλληλα τον παραπάνω κώδικα (copy/paste/amend) ώστε να πάρει την παρακάτω μορφή:

```
1  #include <iostream>
2  #include <string>
3
4  void SplitFilename (const std::string& str)
5  {
6      std::cout << "Splitting: " << str << '\n';
7      unsigned found = str.find_last_of("/\\");
8      std::cout << "  path: " << str.substr(0,found) << '\n';
9      std::cout << "  file: " << str.substr(found+1) << '\n';
10 }
11
12 int main ()
13 {
14     std::string str1 ("/usr/bin/man");
15     std::string str2 ("c:\\windows\\winhelp.exe");
16
17     SplitFilename (str1);
18     SplitFilename (str2);
19
20     std::string str ("Please, erase trailing white-spaces  \n");
21     std::string whitespaces (" \\t\\f\\v\\n\\r");
22
23     std::size_t found = str.find_last_not_of(whitespaces);
24     if (found!=std::string::npos)
25         str.erase(found+1);
26     else
27         str.clear();           // str is all whitespace
28
29     std::cout << "[" << str << "]\n";
30
31     return 0;
32 }
```

Εκτελέστε και ελέγξτε το αποτέλεσμα.

Βήμα 4^ο – Άσκηση:

Αντιγράψτε ένα κομμάτι ελληνικού κειμένου από μία ιστοσελίδα (π.χ. από ένα άρθρο από το www.in.gr) σε ένα αντικείμενο τύπου string. Σε ένα δεύτερο αντικείμενο τύπου string και χρησιμοποιώντας το αρχικό string αντικαταστήστε τους Ελληνικούς χαρακτήρες με τους αντίστοιχους αγγλικούς όπως υποδεικνύεται από τον παρακάτω πίνακα ώστε να "μεταφραστεί" σε greeklish. Τέλος συγκρίνεται το μήκος των δύο strings και συγκρίνετε το μέγεθος της μνήμης που καταλαμβάνουν.

α, ά	a
β	b
γ	g
δ	d
ε, έ	e
ζ	z
η, ή	n
θ	8
ι, ί, ῑ, ῖ	i
κ	k
λ	l
μ	m
ν	v
ξ	3
ο, ό	o
π	p
ρ	r
σ	s
τ	t
υ, ύ, ὕ, ῡ	u
φ	f
χ	x
ψ	y
ω, ώ	w

A, Ά	A
B	B
Γ	G
Δ	D
E, Έ	E
Z	Z
H	H
Θ	8
I, Ί, ῑ	I
K	K
Λ	L
M	M
N	N
Ξ	KS
O, Ό	O
Π	P
P	R
Σ	S
Tα	T
Υ, Ύ, Ὑ	Y
Φ	F
X	X
Ψ	PS
Ω, Ώ	W